

Promises in JS

Why We Use Promise?

- In JavaScript code is executed synchronously that is in sequential order.
- **Synchronous** means the code is executed one by one like one line is executed then the next line is executed and so on.
- This means each statement has to wait for the previous statement to finish executing.
- **Asynchronous operation** in JavaScript breakup long operation into shorter ones so other operations can execute.
- Asynchronous delayed execution by postponing the heavy operation to the end of the event loop, to give event handlers the ability to respond.
- Asynchronous code takes statements outside of the main program flow, allowing the code to run without blocking and waiting for the result or other to finish.

What is Promise?

- Promise is a good way to handle asynchronous operations
- It is used to find out if the asynchronous operation is successfully completed or not.
- A promise may have one of three states
 1. **Pending** – process is not complete
 2. **Fulfilled** – operation successful
 3. **Rejected** – an error occurs

Promise

- In JavaScript promise is the same as a promise we make in real life. When we make a promise in real life, it is assured that we are going to do keep in the future. As promises can only be made for the future.
 - A promise has 2 general outputs: it will either be kept when the time comes, or it won't.
 - Suppose you are appearing for the exam; your dad promises you to give the new bike after getting a pass with first class. That is Promise, A promise has 3 states
1. Pending: You don't know if you will get the bike
 2. Fulfilled: Dad is happy with your first class and he will give you the new bike
 3. Rejected: Dad is not happy as you didn't get the first class, he withholds the new bike

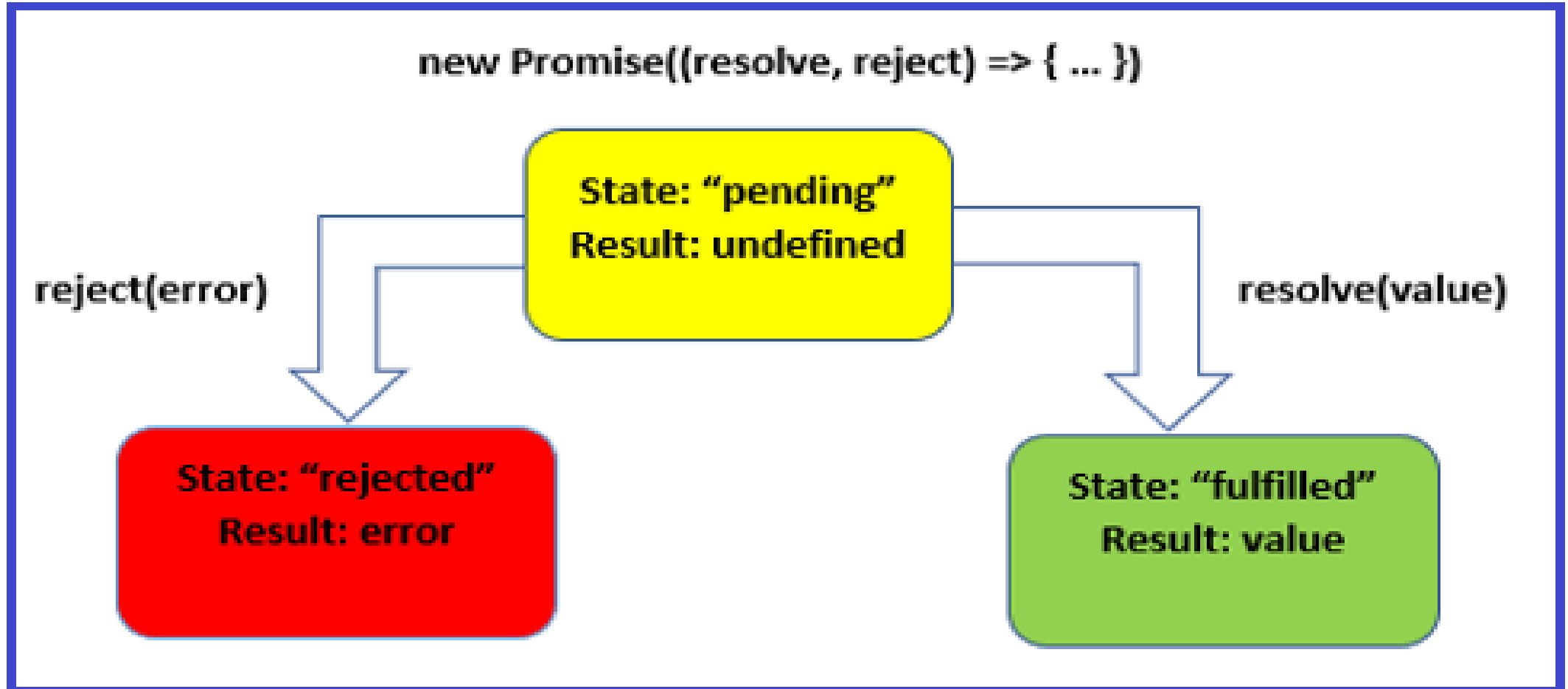
Where to use Promises

- When we are using API and want to make an asynchronous call to remote web services and requesting some data from web service then the JavaScript promise promises us to get data from web services and we can use it in the future.
- It is used for handling asynchronous events and It's mostly used for handling the asynchronous HTTP requests

Promises features

- It's good for handling asynchronous operation
- Good for handling multiple asynchronous operations
- Provide better error handling
- It improves the code readability

The JavaScript Promises States and Control flow



Create A Promise

- To Create a promise object, we use the Promise() constructor
- If the promise returns successfully, the resolve() function is called.
- If an error occurs, the reject() function is called.

```
let promise = new Promise(function(resolve, reject){  
    //do something  
});
```

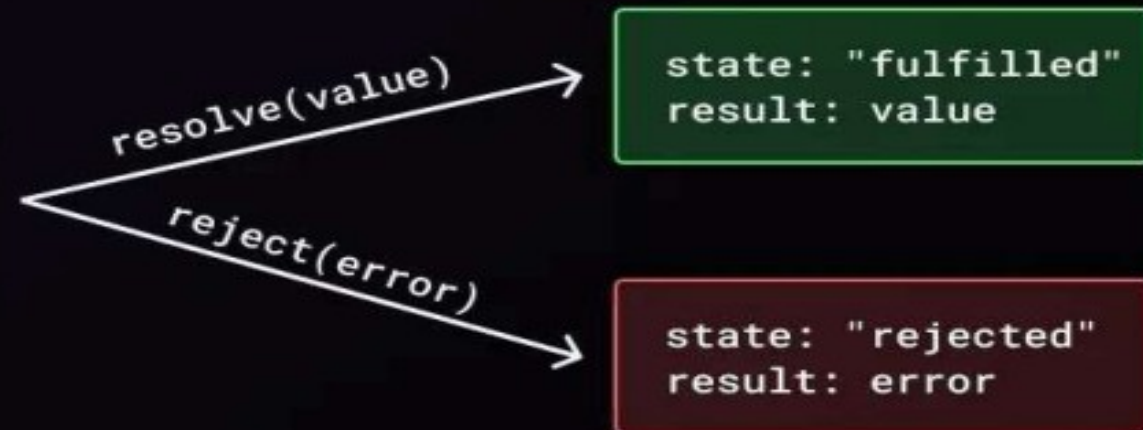
```
const count = true;

let countValue = new Promise(function (resolve, reject) {
  if (count) {
    resolve("There is a count value.");
  } else {
    reject("There is no count value");
  }
});

console.log(countValue);

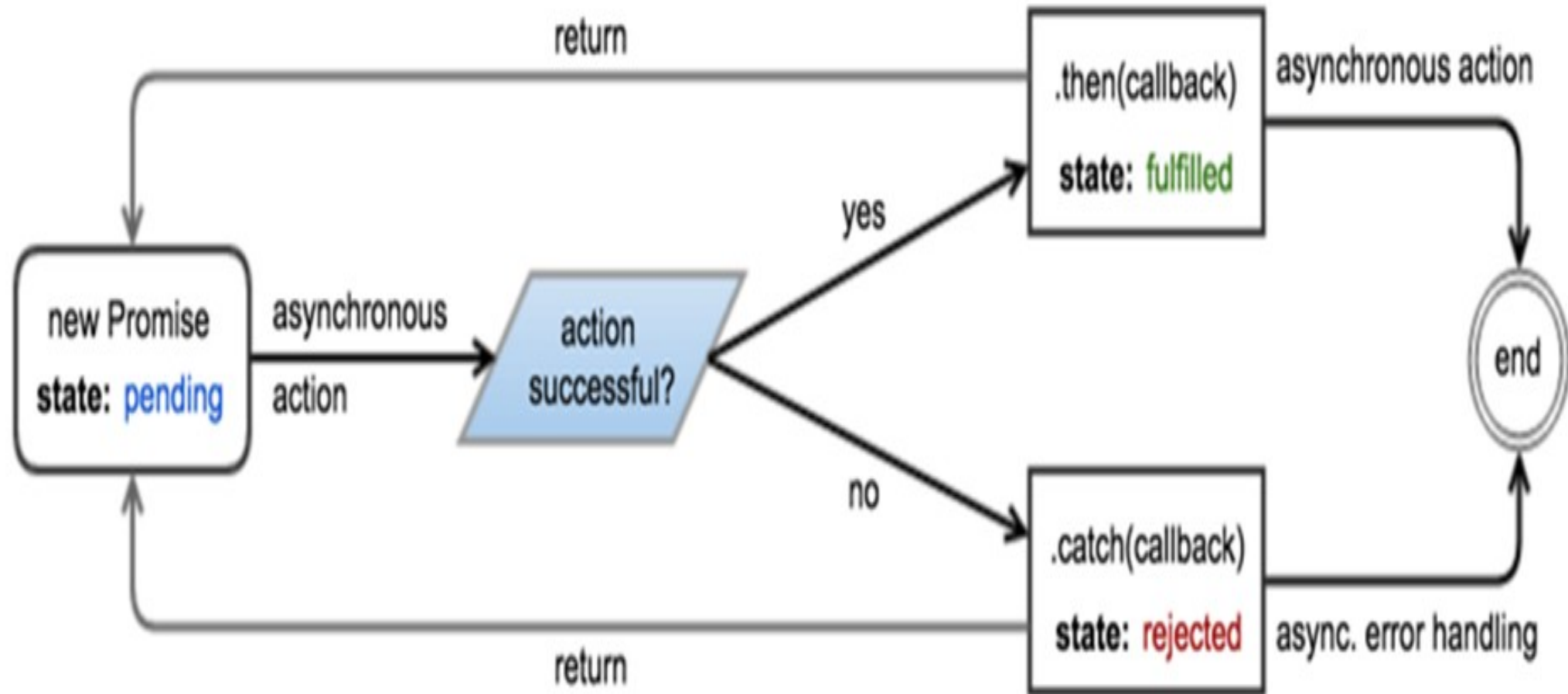
// Promise {<resolved>: "There is a count value."}
```

state: "pending"
result: undefined



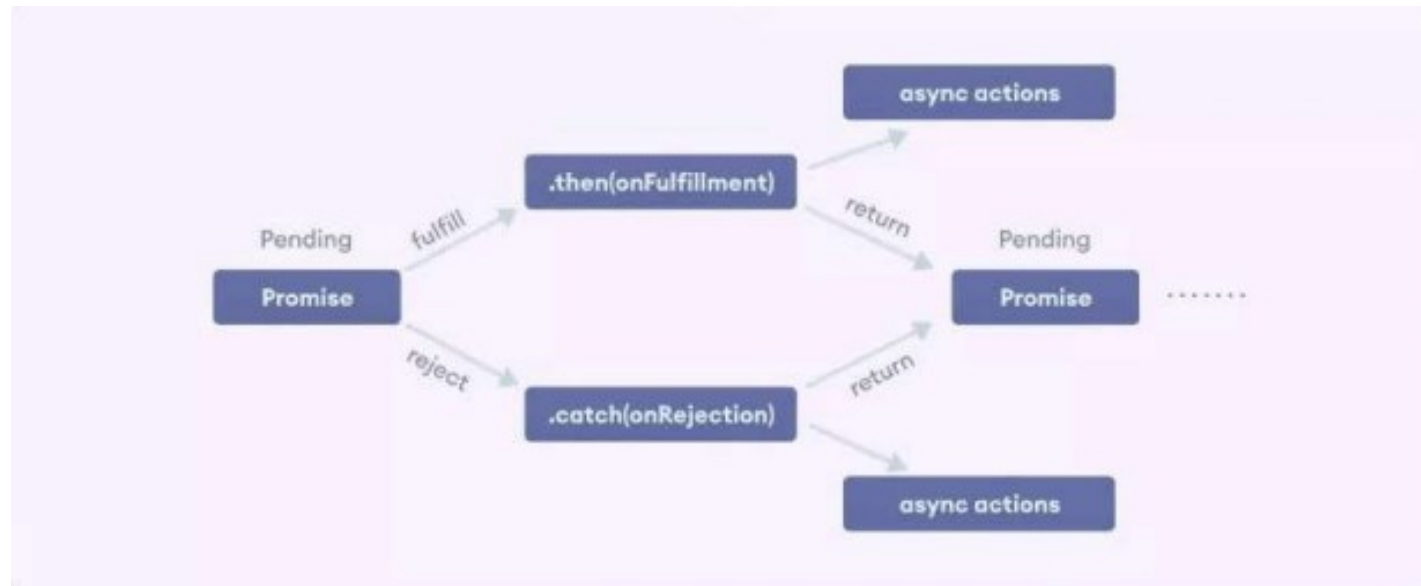
JavaScript Promise Control flow

- A promise is said to be settled or resolved when it is either fulfilled or rejected.
- Once a promise is settled, it becomes immutable, and its state cannot change.
- The `then()` and `catch()` methods of a promise can be used to attach callbacks that execute when it is settled.
- These callbacks are called with the fulfillment value and rejection reason, respectively.



Promise Chaining

- Promises are useful when you have to handle more than one asynchronous task, one after another. For that we use promise chaining.
- To perform an operation after a promise resolved using methods `then()`, `catch()`, and `finally()`



then() Method

The `then()` method is used with the `callback` when the promise is `successfully fulfilled` or `resolved`.

```
promiseObject.then(onFulfilled, onRejected);
```

You can chain multiple `then()` methods with the promise.

```
// returns a promise
let countValue = new Promise(function (resolve, reject) {
  resolve("Promise resolved");
});

// executes when promise is resolved successfully
countValue
  .then(function successValue(result) {
    console.log(result);
  })
  .then(function successValue1() {
    console.log("You can call multiple functions this way.");
  });

// Promise resolved
// You can call multiple functions this way.
```

catch() Method

The `catch()` method is used with the `callback` when the promise is `rejected` or if an `error` occurs.

```
// returns a promise
let countValue = new Promise(function (resolve, reject) {
  reject('Promise rejected');
});

// executes when promise is resolved successfully
countValue
  .then(
    function successValue(result) {
      console.log(result);
    },
  )
// executes if there is an error
  .catch(
    function errorValue(result) {
      console.log(result);
    }
  );

// Promise rejected
```

Promises VS Callback

JavaScript Promise

- The syntax is **user-friendly** and easy to read.
- Error handling is **easier to manage**.

```
api().then(function(result) {  
    return api2();  
}).then(function(result2) {  
    return api3();  
}).then(function(result3) {  
    // do work  
}).catch(function(error) {  
    //handle any error that  
    //may occur before this point  
});
```

JavaScript Callback

- The syntax is **difficult to understand**
- Error handling may be **hard to manage**

```
api(function(result){  
    api2(function(result2){  
        api3(function(result3){  
            // do work  
            if(error) {  
                // do something  
            }  
            else {  
                // do something  
            }  
        });  
    });  
});
```

Task

- Fetch data from this : dummyjson.com/carts and Design a Cart Page using bootstrap.